claude-wsl-ubuntu / 4f364eb7-a600-4a2c-9b37-e5a313b6db9d

Metadata

- Source: `claude-wsl-ubuntu`
- Kind: `claude`
- Source file: `\\wsl.localhost\Ubuntu\home\avidullu\.claude\projects\-home-avidullu-projects-khelsutra-guru\4f364eb7-a600-4a2c-9b37-e5a313b6db9d\subagents\workflows\wf_fa7a2077-3a9\agent-a0dbe813153f9151d.jsonl`
- SHA-256: `2a6574464f34790cc243418cda0adb7084fa13b01a0e3f7944eeff50d6d5ea3f`
- Source modified: `2026-06-29T10:52:26+00:00`
- Imported at: `2026-07-05T16:48:30+00:00`
- project: `wf_fa7a2077-3a9`
- session_id: `4f364eb7-a600-4a2c-9b37-e5a313b6db9d`

Transcript

1. user (2026-06-29T10:50:03.925Z)

In /home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer: read run_all_tests.py and figure out what it runs. Search the tests/ tree and pytest config (pytest.ini / pyproject.toml / setup.cfg / conftest.py) for how GPU, real-video, golden-corpus, and native-WASB tests are gated (pytest markers, skipif conditions, env vars). Read CLAUDE.md and list its hard guardrails (commercial-clean licensing, paid-LLM = inference only, owner-gated work, branch/PR rules). Report the ruff and mypy invocation/config and any coverage floor enforced in CI (.github/workflows). State the expected passing test count if documented (~1487+). Return data, not prose.

2. assistant (2026-06-29T10:50:06.179Z)

I'll analyze the badminton-highlight-indexer project. Let me start by reading the key files.

3. user (2026-06-29T10:50:07.226Z)

```
total 192
drwxr-xr-x 22 avidullu avidullu  4096 Jun 27 17:27 .
drwxr-xr-x 10 avidullu avidullu  4096 Jun 29 09:48 ..
-rw-r--r--  1 avidullu avidullu   598 Jun 25 11:46 .coveragerc
drwxr-xr-x  8 avidullu avidullu  4096 Jun 29 10:44 .git
-rw-r--r--  1 avidullu avidullu   932 Jun 25 11:46 .gitattributes
drwxr-xr-x  3 avidullu avidullu  4096 Jun 25 11:46 .github
-rw-r--r--  1 avidullu avidullu  1331 Jun 25 16:06 .gitignore
drwxr-xr-x  3 avidullu avidullu  4096 Jun 25 13:12 .mypy_cache
drwxr-xr-x  3 avidullu avidullu  4096 Jun 25 13:10 .pytest_cache
drwxr-xr-x  3 avidullu avidullu  4096 Jun 25 13:12 .ruff_cache
drwxr-xr-x  2 avidullu avidullu  4096 Jun 25 11:46 .vscode
-rw-r--r--  1 avidullu avidullu  5498 Jun 27 17:27 CLAUDE.md
-rw-r--r--  1 avidullu avidullu 34880 Jun 25 11:46 README.md
drwxr-xr-x  2 avidullu avidullu  4096 Jun 25 13:10 __pycache__
drwxr-xr-x 20 avidullu avidullu  4096 Jun 27 17:27 backend
-rw-r--r--  1 avidullu avidullu  2650 Jun 25 11:46 config.json
drwxr-xr-x  5 avidullu avidullu  4096 Jun 25 11:46 deploy
drwxr-xr-x  6 avidullu avidullu  4096 Jun 29 10:44 docs
drwxr-xr-x  3 avidullu avidullu  4096 Jun 25 11:46 eval_baselines
drwxr-xr-x  3 avidullu avidullu  4096 Jun 25 13:11 gcs:
drwxr-xr-x  4 avidullu avidullu  4096 Jun 25 13:11 gs:
drwxr-xr-x  3 avidullu avidullu  4096 Jun 25 13:11 local:
-rw-r--r--  1 avidullu avidullu  1960 Jun 25 11:46 mypy.ini
drwxr-xr-x  3 avidullu avidullu  4096 Jun 25 18:13 output
-rw-r--r--  1 avidullu avidullu  4748 Jun 27 17:27 pyproject.toml
-rw-r--r--  1 avidullu avidullu   240 Jun 25 11:46 pytest.ini
-rw-r--r--  1 avidullu avidullu  2003 Jun 27 17:27 requirements.txt
-rw-r--r--  1 avidullu avidullu   789 Jun 25 11:46 ruff.toml
```

```
-rw-r--r--  1 avidullu avidullu  2051 Jun 27 17:27 run_all_tests.py
-rw-r--r--  1 avidullu avidullu  5572 Jun 27 17:27 run_eval.py
-rw-r--r--  1 avidullu avidullu  4457 Jun 27 17:27 run_regression.py
drwxr-xr-x  2 avidullu avidullu  4096 Jun 25 11:46 scratch
drwxr-xr-x  3 avidullu avidullu  4096 Jun 27 17:27 scripts
drwxr-xr-x  3 avidullu avidullu  4096 Jun 29 10:44 tests
drwxr-xr-x  3 avidullu avidullu  4096 Jun 27 17:27 tools
drwxr-xr-x  4 avidullu avidullu  4096 Jun 25 13:10 training
```

4. user (2026-06-29T10:50:07.693Z)

```
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.

A single, discoverable entry point so you (or a reviewer / CI) can verify the
whole suite before opening a PR:

    python run_all_tests.py             # full suite, quiet
    python run_all_tests.py -v          # verbose (passthrough to pytest)
    python run_all_tests.py -k config   # filter by keyword (passthrough)
    python run_all_tests.py --cov=backend --cov-report=term-missing   # coverage

Anything after the script name is passed straight through to pytest.

Notes
-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional ``obsreport``
  package isn't installed (they use ``pytest.importorskip``). To exercise them too:
      pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""

from __future__ import annotations

import importlib.util
import subprocess
import sys


def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False


def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
        args = ["-q", *args]
    cmd = [sys.executable, "-m", "pytest", *args]

    obs = (
        "available"
        if _installed("obsreport")
        else "NOT installed ? reporting tests will SKIP"
    )
    bar = "=" * 72
    print(bar)
    print(" Full test suite:  " + " ".join(cmd))
    print(f" obsreport (optional reporting dep): {obs}")
    print(bar, flush=True)

```

```
55          rc = subprocess.run(cmd).returncode
56
57          print(bar)
58          print(
59              " RESULT: "
60              + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit {rc}")
61          )
62          print(bar)
63          return rc
64
65
66      if __name__ == "__main__":
67          raise SystemExit(main())
68
```

5. assistant (2026-06-29T10:50:10.145Z)

Let me read the pytest config files, CLAUDE.md, and explore the tests tree.

6. user (2026-06-29T10:50:10.855Z)

```
1       [pytest]
2       # When `pytest` is invoked without an explicit path, only collect from tests/.
3       # scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.)
4       # and should never run as part of the regular test suite.
5       testpaths = tests
6
```

7. user (2026-06-29T10:50:11.325Z)

```
1       # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2       #
3       # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4       # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5       # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6       # auto-detection would otherwise miss the un-__init__'d `backend` root.
7       #
8       # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9       # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10      # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11      # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12      [build-system]
13      requires = ["hatchling"]
14      build-backend = "hatchling.build"
15
16      [project]
17      name = "badminton-highlight-indexer"
18      version = "0.1.0"
19      description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20      readme = "README.md"
21      requires-python = ">=3.10"
22      license = { text = "MIT" }
23      authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24      keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26      # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27      dependencies = [
28          "fastapi>=0.111.0",
29          "uvicorn>=0.30.1",
30          "opencv-python>=4.9.0.80",
31          "numpy>=1.26.4",
32          "pydantic>=2.7.1",
33          "jinja2>=3.1.4",
```

```
34          "python-dotenv>=1.0.0",
35          "google-genai>=2.4.0",
36          "supervision>=0.21.0",
37          # Used directly on the serving path (backend/features/rally_seq.py: uniform_filter1d
/
38          # maximum_filter1d); previously present only TRANSITIVELY via supervision, so a
supervision
39          # bump or a pruned-transitive freeze could silently break inference. Declare it.
(BSD-3.)
40          "scipy>=1.10",
41          # D13/P12: ByteTrack moved to the standalone `trackers` package (Apache-2.0). Its
42          # PyPI wheel is broken (metadata-only), so it MUST be installed from git ? see
43          # requirements.txt. PEP 621 cannot express a git URL, so it's NOT listed here
44          # (same pattern as torch/torchvision). CI installs it on its own line.
45      ]
46
47      [project.optional-dependencies]
48      # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
49      dev = [
50          "pytest",
51          "pytest-cov",
52          "httpx",        # fastapi TestClient transport
53          "ruff",
54          "mypy",
55          "PyJWT[crypto]>=2.8",  # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
56      ]
57
58      # M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when
security.edge_auth_enabled=True.
59      # `jwt` is lazy-imported, so the base runtime works without this; install it for
hosted/multi-tenant.
60      auth = ["PyJWT[crypto]>=2.8"]
61
62      # Per-video observability reports (SOFT dependency: the pipeline degrades to a
63      # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
64      # fetched over SSH; if you lack access, simply omit this group ? reporting
65      # tests skip via pytest.importorskip. For local dev against a sibling checkout:
66      #   pip install -e ../sports-obsreport --no-build-isolation
67      reporting = [
68          "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
69      ]
70
71      # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
72      # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
73      # express, so they are NOT declarable here. Install them out-of-band instead:
74      #   # CUDA 12.4 box:
75      #   pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
76      #   # CPU-only box (also what CI uses):
77      #   pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
78      gpu = []
79
80      # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
81      # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
82      # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
83      # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
84      # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
85      storage-s3 = ["boto3>=1.34"]
86      storage-gcs = ["google-cloud-storage>=2.16"]
87
88      [project.scripts]
89      # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
90      indexer = "backend.main:main"
```

```
91
92      [tool.hatch.metadata]
93      # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
94      # reference*); hatchling requires opting in to allow those in metadata.
95      allow-direct-references = true
96
97      [tool.hatch.build.targets.wheel]
98      # `backend` is a namespace package (no __init__.py); list both import roots
99      # explicitly so an editable/wheel install exposes `import backend` and
100     # `import training` regardless of cwd.
101     packages = ["backend", "training"]
102
```

8. user (2026-06-29T10:50:13.555Z)

```
1       # Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would
2       # bury blame history for zero behavior value).
3       #
4       # Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle
5       # warnings (W) + bugbear (B) + isort (I ? consistent import ordering).
6       target-version = "py38"
7
8       # scratch/ = throwaway debugging scripts that may hit real APIs; it is also
9       # excluded from pytest collection (see pytest.ini).
10      extend-exclude = ["scratch"]
11
12      [lint]
13      extend-select = ["W", "B", "I"]
14
15      [lint.per-file-ignores]
16      # Entrypoint scripts deliberately run setup (load_dotenv, logging config,
17      # sys.path) before importing backend modules ? import order is load-bearing.
18      "backend/main.py" = ["E402"]
19      "scripts/run_phase3_eval.py" = ["E402"]
20      "scripts/run_process_and_stitch.py" = ["E402"]
21
```

9. user (2026-06-29T10:50:13.960Z)

```
1       [mypy]
2       # Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean
3       # modules and pins an explicit quarantine list below ? not a typing crusade.
4       # Ratchet: when you touch a quarantined module, try removing its entry.
5       ignore_missing_imports = True
6       # backend/ is a namespace package (no root __init__.py) ? required so mypy
7       # maps files to backend.* module names instead of duplicating top-level names.
8       explicit_package_bases = True
9
10      # --- Quarantine (known-loose modules, each with a reason) ---
11
12      # Optional module globals (db_instance/global_config set by create_app() / main()).
13      # Route handlers use request.app.state exclusively; helpers take explicit params.
14      # backend.api.job_worker lifted from quarantine (now takes explicit config/db params).
15      [mypy-backend.main]
16      ignore_errors = True
17
18      # Worker dispatcher: composes the SAME validator/segmenter/report building blocks as
19      # backend.main, which are typed `config: dict` but receive the AppConfig migration shim
20      # at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine
21      # until the typed-config migration removes the dict?AppConfig duality everywhere.
22      [mypy-backend.worker.__main__]
23      ignore_errors = True
24
25      # yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code
26      # to detectors/person_detector.py (which IS type-clean, no quarantine). What remains
```

```
27      # here is pre-existing orchestration debt unrelated to that move (implicit-Optional
28      # process_video defaults, the Success|Failure ABC-override mismatch, a Future[...]
29      # annotation) ? ratchet those out in a focused typing pass, not here.
30      # (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)
31      [mypy-backend.pipeline.segmenters.yolo_hybrid]
32      ignore_errors = True
33
34      [mypy-backend.pipeline.segmenters.gemini]
35      ignore_errors = True
36
37      # Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.
38      [mypy-backend.pipeline.segmenters.base]
39      ignore_errors = True
40
```

10. user (2026-06-29T10:50:14.452Z)

```
1       [run]
2       # Excluded from coverage: the legacy motion segmenter (demo baseline with
3       # fabricated metadata) and the concrete validators (thin OpenCV/ffprobe
4       # wrappers only exercisable against real media files). validators/base.py
5       # stays counted ? it has a dedicated unit suite.
6       omit =
7           backend/pipeline/segmenters/motion.py
8           backend/pipeline/validators/format.py
9           backend/pipeline/validators/resolution_fps.py
10          backend/pipeline/validators/pts_continuity.py
11          backend/pipeline/validators/scene_cut.py
12          backend/pipeline/validators/blur.py
13          backend/pipeline/validators/relevance.py
14
```

11. assistant (2026-06-29T10:50:17.477Z)

Now let me explore the tests tree, conftest, and CI workflows.

12. user (2026-06-29T10:50:21.148Z)

```
---TREE---
/home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer/tests
/home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer/tests/__pycache__
---COUNT FILES---
129
```

13. user (2026-06-29T10:50:21.605Z)

```
total 36
drwxr-xr-x 2 avidullu avidullu  4096 Jun 27 17:27 .
drwxr-xr-x 3 avidullu avidullu  4096 Jun 25 11:46 ..
-rw-r--r-- 1 avidullu avidullu 10871 Jun 27 17:27 ci.yml
-rw-r--r-- 1 avidullu avidullu 12752 Jun 27 17:27 nightly.yml
```

14. assistant (2026-06-29T10:50:23.953Z)

There's no conftest.py in tests/. Let me look for markers, skipif gating, and read the CI workflows. Let me search the test tree for gating mechanisms.

15. user (2026-06-29T10:50:28.809Z)

```
=== conftest anywhere ===
=== markers / skip / gating in tests ===
tests/test_path_safety.py:12:@pytest.mark.parametrize(
tests/test_path_safety.py:31:@pytest.mark.parametrize("ok", ["highlights.mp4", "my_reel-1.mp4",
"out.MP4"])
```

```
tests/test_native_wasb_runner.py:1:"""NativeWasbRunner coverage ? the Linux-native (no-WSL) WASB
detector path (M3.5).
tests/test_native_wasb_runner.py:3:The native subprocess is mocked (no torch/conda/GPU here).
These tests lock the contract
tests/test_native_wasb_runner.py:4:the GPU-box byte-parity gate then relies on: it IS a
DetectorRunner, it is NOT a WSL runner
tests/test_native_wasb_runner.py:5:(no WslCommandMixin / wsl invocation), it builds a native
argv (no `wsl`, no `conda
tests/test_native_wasb_runner.py:18:from backend.pipeline.detectors.native_wasb_runner import (
tests/test_native_wasb_runner.py:34:    """The whole point of M3.5: the native runner must carry
NO WSL plumbing."""
tests/test_native_wasb_runner.py:56:    assert cfg.repo_dir == "~/models/WASB-SBDT"
tests/test_native_wasb_runner.py:57:    # Native default is STREAMING (perf; bit-identical to
disk, measured on a real GPU).
tests/test_native_wasb_runner.py:75:                "repo_dir": "/m/WASB-SBDT",
tests/test_native_wasb_runner.py:83:    assert cfg.python_bin == "/envs/wasb/bin/python" and
cfg.repo_dir == "/m/WASB-SBDT"
tests/test_native_wasb_runner.py:90:    monkeypatch.setenv("WASB_WEIGHTS", "/env/w.pth.tar")
tests/test_native_wasb_runner.py:91:    monkeypatch.setenv("WASB_DEVICE", "mps")
tests/test_native_wasb_runner.py:92:    monkeypatch.setenv("WASB_PYTHON", "/env/py")
tests/test_native_wasb_runner.py:93:    monkeypatch.setenv("WASB_REPO_DIR", "/env/repo")
tests/test_native_wasb_runner.py:94:    monkeypatch.setenv("WASB_SCRATCH", "/env/scratch")
tests/test_native_wasb_runner.py:173:    assert ok  # cpu run does not require a GPU
tests/test_native_wasb_runner.py:202:    assert r._effective_device() == "cuda"  # auto resolves
to the GPU when present
tests/test_native_wasb_runner.py:206:    """device=auto on a GPU-less box does NOT hard-fail
(unlike device=cuda) ? it resolves to cpu
tests/test_native_wasb_runner.py:207:    and warns loudly. This is the M4 portability win for a
no-GPU Linux / cpu-serving box."""
tests/test_native_wasb_runner.py:212:        logging.WARNING,
logger="backend.pipeline.detectors.native_wasb_runner"
tests/test_native_wasb_runner.py:215:    assert ok  # auto does not require a GPU
tests/test_native_wasb_runner.py:233:    """run_predict with device=auto + no GPU: the lazy
probe (mocked empty stdout = no cuda)
tests/test_native_wasb_runner.py:243:    """run_predict with device=auto + a GPU: the probe
reports cuda True ? --device cuda."""
tests/test_native_wasb_runner.py:291:def test_run_predict_builds_native_argv_no_wsl(tmp_path):
tests/test_native_wasb_runner.py:348:    """Unmocked: the native runner stages the SAME
wasb_infer.py the WSL runner runs (the
tests/test_native_wasb_runner.py:352:    from backend.pipeline.detectors import
native_wasb_runner as nwr
tests/test_native_wasb_runner.py:363:
"backend.pipeline.detectors.native_wasb_runner.shutil.copy",
tests/test_native_wasb_runner.py:432:    """The native CSV name must equal the WSL runner's so
downstream + the parity diff line up."""
tests/test_native_wasb_runner.py:439:# --- the detector_impl='native' seam (parallels the stub
seam) --------------------------
tests/test_native_wasb_runner.py:440:def
test_resolve_runner_native_selects_native_runner_for_wasb(monkeypatch):
tests/test_native_wasb_runner.py:445:        {"detector_impl": "native"}
tests/test_native_wasb_runner.py:448:    assert params.get("detector_impl") == "native"
tests/test_native_wasb_runner.py:451:def
test_resolve_runner_native_env_overrides_config(monkeypatch):
tests/test_native_wasb_runner.py:452:    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
tests/test_native_wasb_runner.py:459:def
test_resolve_runner_native_supported_for_fusion_wasb(monkeypatch):
tests/test_native_wasb_runner.py:464:        {"detector_impl": "native"}
tests/test_native_wasb_runner.py:470:def
test_resolve_runner_native_refused_for_fusion_tracknet(monkeypatch):
tests/test_native_wasb_runner.py:476:            {"detector_impl": "native", "fusion":
{"substrate": "tracknet"}}
tests/test_native_wasb_runner.py:480:def
test_resolve_runner_native_refused_for_tracknet(monkeypatch):
tests/test_native_wasb_runner.py:485:        TrackNetHybridSegmenter(None,
{})._resolve_runner({"detector_impl": "native"})
```

```
tests/test_worker.py:4:segmenter mocked, so it needs no real video/GPU/network. The real S3
round-trip is validated
tests/test_worker.py:18:@pytest.mark.parametrize(
tests/test_worker.py:222:    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
tests/test_worker.py:247:    assert "detector_impl=native" in msgs  # surfaces the resolved
detector
tests/test_worker.py:248:    assert "native runner UNAVAILABLE" in msgs  # points at the likely
cause
tests/test_worker.py:288:@pytest.mark.parametrize(
tests/test_worker.py:321:    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # CPU stub
runner (no GPU/WSL)
tests/test_worker.py:662:    RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the
stub IS the runner)."""
tests/test_worker.py:668:    )  # resolve a CPU stub runner (no GPU/WSL)
tests/test_worker.py:732:def
test_cmd_train_detector_native_on_tracknet_fails_clean_not_traceback(
tests/test_worker.py:735:    """detector_impl=native for a family with no native runner must
return rc 2 (clean), not
tests/test_worker.py:737:    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
tests/test_edge_auth.py:14:pytest.importorskip(
tests/test_edge_auth.py:87:@pytest.mark.parametrize(
tests/test_edge_auth.py:195:@pytest.mark.parametrize("team,aud", [("   ", AUD), (TEAM, "  "),
("\t", "\n")])
tests/test_edge_auth.py:218:pytest.importorskip("httpx")  # fastapi TestClient transport
tests/test_serve_contrast.py:3:GPU-free, synthetic trajectories + golden labels written to tmp
files. Pins:
tests/test_fusion_features.py:3:No GPU/torch/cv2 ? synthetic person boxes + shuttle points
exercise every feature path and
tests/test_fusion_features.py:4:edge case. This is the testable core that proves the fusion math
correct before any GPU run.
tests/test_input_source.py:24:@pytest.mark.parametrize(
tests/test_ffmpeg_resolver.py:60:@pytest.mark.parametrize(
tests/test_output_base.py:97:    ) as preg, patch("os.makedirs"), patch("os.environ.get",
return_value="key"):
tests/test_person_detector.py:4:without GPU/torch weights. They pin the COCO person==1 filter
(the YOLO-was-0 gotcha) and
tests/test_person_detector.py:215:    pytest.importorskip("trackers")
tests/test_person_detector.py:270:    pytest.importorskip("trackers")
tests/test_person_detector.py:290:    pytest.importorskip("trackers")
tests/test_storage.py:31:@pytest.mark.parametrize(
tests/test_rally_seg_eval.py:123:    pytest.importorskip(
tests/test_rally_seg_eval.py:229:    # known multicourt clip (from GOLDEN_VIDEOS.md /
MULTICOURT_CLIPS)
tests/test_bind_and_host.py:5:pytest.importorskip("httpx")  # TestClient needs httpx
tests/test_fusion_golden_telemetry.py:1:"""GPU-free tests for the run-telemetry wire-in to the
fusion_golden extractor.
tests/test_fusion_golden_telemetry.py:4:around each extract_video; a hard kill during a long GPU
run thus leaves a forensic runlog. These
tests/test_fusion_golden_telemetry.py:5:tests stub extract_video (no GPU) and inject
deterministic collectors + clock via a patched
tests/test_fusion_golden_telemetry.py:39:    fixed GPU-name header (no real nvidia-smi). Returns
nothing; run() builds the instance itself."""
tests/test_fusion_golden_telemetry.py:40:    monkeypatch.setattr(rt, "gpu_name", lambda:
"FakeGPU")
tests/test_fusion_golden_telemetry.py:119:        raise RuntimeError("GPU vanished")
tests/test_fusion_golden_telemetry.py:128:        assert "GPU vanished" in str(e)
tests/test_fusion_golden_telemetry.py:137:    assert errs[0]["msg"] == "alpha" and "GPU
vanished" in errs[0]["err"]
```

16. assistant (2026-06-29T10:50:31.834Z)

No conftest.py and no pytest markers registered. The gating is done via `pytest.importorskip` and `monkeypatch.setenv`. Let me look more specifically at how golden-corpus, real-video, and GPU tests are gated, and check for any `skipif`/env-based skips. Let me also read the CI workflows.

17. user (2026-06-29T10:50:38.769Z)

```
Exit code 2
=== importorskip occurrences ===
tests/test_ui_bundle.py:8:pytest.importorskip("httpx")
tests/test_reporting.py:80:# (see test_reporting_harvest.py which uses pytest.importorskip).
Those tests
tests/test_edge_auth.py:14:pytest.importorskip(
tests/test_edge_auth.py:218:pytest.importorskip("httpx")  # fastapi TestClient transport
tests/test_bind_and_host.py:5:pytest.importorskip("httpx")  # TestClient needs httpx
tests/test_reporting_spec_faq.py:14:pytest.importorskip("obsreport")
tests/test_wasb_cache.py:368:    cv2 = pytest.importorskip("cv2")  # noqa: F841 (importorskip
gates the suite on cv2)
tests/test_wasb_cache.py:375:    cv2 = pytest.importorskip("cv2")
tests/test_wasb_cache.py:401:    pytest.importorskip("cv2")
tests/test_wasb_cache.py:418:    cv2 = pytest.importorskip("cv2")
tests/test_wasb_cache.py:419:    Image = pytest.importorskip("PIL.Image")
tests/test_ablation.py:400:    pytest.importorskip("reportlab")
tests/test_ablation.py:401:    pypdf = pytest.importorskip("pypdf")
tests/test_ablation.py:629:    pytest.importorskip("reportlab")
tests/test_ablation.py:630:    pytest.importorskip("pypdf")
tests/test_import_smoke.py:110:    pytest.importorskip("numpy")
tests/test_import_smoke.py:111:    pytest.importorskip("scipy")
tests/test_upload_download.py:15:pytest.importorskip("httpx")  # TestClient needs httpx; skip
cleanly where absent
tests/test_nightly_reelcount.py:276:    pytest.importorskip("cv2")
tests/test_reporting_harvest.py:11:pytest.importorskip("obsreport")
tests/test_job_worker_loop.py:26:pytest.importorskip("httpx")
tests/test_api_cloud_dispatch.py:16:pytest.importorskip("httpx")
tests/test_served_gate_a_regression.py:47:pytest.importorskip("cv2")
tests/test_async_jobs.py:20:pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly
where absent
tests/test_person_detector.py:215:    pytest.importorskip("trackers")
tests/test_person_detector.py:270:    pytest.importorskip("trackers")
tests/test_person_detector.py:290:    pytest.importorskip("trackers")
tests/test_rally_seg_eval.py:123:    pytest.importorskip(
tests/test_api_endpoints.py:11:pytest.importorskip("httpx")  # TestClient needs httpx; skip
cleanly where absent

=== skipif / pytest.skip occurrences ===
tests/test_app_home.py:193:        pytest.skip("Windows drive-letter semantics only meaningful
on nt")
tests/test_config_deployment.py:401:        pytest.skip("no repo config.json")
tests/test_served_gate_a_regression.py:82:_skip = pytest.mark.skipif(
tests/test_ablation.py:525:@pytest.mark.skipif(
tests/test_ablation.py:555:@pytest.mark.skipif(

=== markers registered in any config ===
```

18. assistant (2026-06-29T10:50:41.171Z)

Now let me look at the specific skipif conditions for golden/real-video gating, and read the CI workflows.

19. user (2026-06-29T10:50:43.838Z)

```
=== test_served_gate_a_regression skipif context ===
import json
import os

import pytest
```

cv2 is imported transitively by trajectory_hybrid (the served engine). On a box without

(CI), skip the whole module rather than erroring at import.

```
pytest.importorskip("cv2")

from backend.eval import segmentation_metrics as _segm  # noqa: E402
from backend.eval import served_gate_a as sga  # noqa: E402
from backend.eval.calibrate_wasb import load_golden_gts  # noqa: E402
from backend.eval.metrics import score  # noqa: E402
from backend.pipeline.detectors import rally_gate  # noqa: E402
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner  # noqa: E402
```

The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a fresh worktree / CI). Resolve it relative to this repo root first, with an env override for an out-of-tree checkout.

```
_REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
MANIFEST = os.environ.get(
    "SERVED_GATE_A_MANIFEST",
    os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
)


def _golden_specs():
    """Return the manifest specs whose trajectory + label files are all present, else []."""
    if not os.path.isfile(MANIFEST):
        return []
    with open(MANIFEST, encoding="utf-8") as f:
        specs = json.load(f)
    ok = [
        s
        for s in specs
        if os.path.isfile(str(s.get("trajectory", "")))
        and os.path.isfile(str(s.get("labels", "")))
    ]
    return ok if len(ok) >= 6 else []


_SPECS = _golden_specs()
_skip = pytest.mark.skipif(
    not _SPECS,
    reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
)
```

---------- served-path litmus

```
@_skip
def test_served_path_runs_on_all_golden_videos():
    """The production FusionSegmenter decision path runs end-to-end (no GPU/Gemini) on EVERY
    present golden trajectory and yields a handoff window list for each, for both gate settings.
    Asserts against the present corpus count (>= 6), not a stale fixed 6 ? the golden set
grows."""
    results = sga.evaluate_manifest(_SPECS)
    assert len(results) == len(_SPECS)
    assert len(results) >= 6
    for r in results:
        assert r.num_gt > 0
=== test_ablation skipif context (520-560) ===
        if _is_fusion_schema(
            cands
        ):  # canonical predicate (SSOT in backend.eval.experiment, #215)
            n += 1
    return n >= 6
```

```
@pytest.mark.skipif(
    not _golden_present(),
    reason="golden output/*_golden_features.json absent (run fusion_golden first)",
)
def test_litmus_reproduces_gate_a():
    """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-measured Gate-A
    result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
tolerance.
    This validates the whole framework against the recorded ground truth. Skips cleanly in CI
    where the golden substrate is absent; runs on the owner box where it is present."""
    videos = ablation.load_videos(GOLDEN_DIR)
    assert len(videos) == 6
    cfg = ablation.litmus_config()
    report = run_ablation(videos, cfg)
    assert len(report.rows) == 1
    row = report.rows[0]
    assert row.signal == "gate_a"
    # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
    assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
    assert row.sign_wins == 6 and row.sign_losses == 0
    assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
    assert row.ci_low == pytest.approx(0.042, abs=0.01)
    assert row.ci_high == pytest.approx(0.104, abs=0.01)
    assert row.ci_excludes_zero is True
    # Gate-A is structural ? reported but never auto-promoted to earns_keep.
    assert row.verdict == ablation.STRUCTURAL_PROTECTED
    # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
    assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
    assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)


@pytest.mark.skipif(
    not _golden_present(), reason="golden output/*_golden_features.json absent"
)
def test_litmus_report_is_well_formed():
    """The litmus report is a proper AblationReport (provenance + a sorted single row)."""
    report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
=== test_rally_seg_eval importorskip (118,130) ===
    assert agg["n"] == 0 and agg["f1"] is None
```

--- W2: net-crossings gate (Gate-A) on the windowing ------------------------------

```
def test_windows_for_netcross_gate(tmp_path):
    pytest.importorskip(
        "cv2"
    )  # rally_gate import path tolerance (no cv2 actually needed)
    video = {
        "trajectory": _write_oscillating_traj(tmp_path / "osc.csv"),
        "fps": 30.0,
        "frame_width": 1280.0,
    }
    ungated = windows_for(video, _PRESET, min_crossings=0)
    assert len(ungated) >= 1  # a window forms
    # The oscillating shuttle crosses the net many times ? survives a modest gate...
    assert len(windows_for(video, _PRESET, min_crossings=2)) >= 1
    # ...but an absurd crossing requirement that would gate EVERY window out instead falls
```

20. assistant (2026-06-29T10:50:46.864Z)

Now let me read the CI workflows to find ruff/mypy invocation, coverage floor, and the expected test count.

21. user (2026-06-29T10:50:47.832Z)

```yaml
name: CI

on:
  push:
    branches: [master]
  pull_request:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
  # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
  # tracked .py, including scripts/ and training/, which the backend-only
  # py_compile smoke sweep does not reach.
  lint:
    name: Ruff lint
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install ruff
        run: python -m pip install ruff
      - name: Ruff check
        run: ruff check . --output-format github

  # PII / attribution leak guard (tools/leak_scan.py): fails on a committed 32-hex MD5
  # (the video_id shape) or a personal absolute user path in the shipped-code surface
  # (backend/training/scripts/tools/eval_baselines; tests/ excluded ? legit fixtures).
Real
  # names are scanned only via a gitignored private list locally; CI runs the structural
guard.
  leak-scan:
    name: Leak scan (PII / attribution guard)
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Leak scan
        run: python -m tools.leak_scan

  # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
  # per-module quarantine list ? ratchet by removing entries as modules clean
  # up. Typed core deps are installed so mypy sees their real signatures
  # (without them ignore_missing_imports would Any-stub half the checks).
  typecheck:
    name: Mypy (lenient baseline)
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install mypy + typed core deps
```

```
64              run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
65            - name: Mypy
66              run: mypy backend training
67
68        # Fast guard against the 7fbbeb0 class of breakage: a SyntaxError or
69        # module-level import error shipping while the suite stays green.
70        # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
71        # on machines without the GPU stack (test_import_smoke stubs what's absent).
72        import-smoke:
73          name: Syntax sweep + import smoke (no GPU stack)
74          runs-on: ubuntu-latest
75          timeout-minutes: 10
76          steps:
77            - uses: actions/checkout@v6
78            - uses: actions/setup-python@v6
79              with:
80                python-version: "3.13"
81            - name: Install minimal deps
82              run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
83            - name: Run import smoke tests
84              run: python -m pytest tests/test_import_smoke.py -v
85
86        full-suite:
87          name: Full test suite (offline, mocked)
88          runs-on: ubuntu-latest
89          timeout-minutes: 30
90          steps:
91            - uses: actions/checkout@v6
92            - uses: actions/setup-python@v6
93              with:
94                python-version: "3.13"
95                cache: pip
96            - name: System libs for opencv-python
97              run: |
98                # Drop the runner's pre-installed Microsoft apt repo (azure-cli): it
periodically
99                # 403s on InRelease and aborts `apt-get update` under the `-e` shell, failing
the
100               # build on an unrelated repo we don't use. libgl1/libglib2.0 are in Ubuntu's
archive.
101               sudo rm -f /etc/apt/sources.list.d/*microsoft* /etc/apt/sources.list.d/*azure*
102               sudo apt-get update
103               sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
104           - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
105             # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
106             # need an out-of-band index that PEP 621 can't express. Keep installing
107             # them separately here, before the editable install pulls the rest.
108             run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
109           - name: Install trackers (git-only dep ? PyPI wheel is broken)
110             # D13/P12: ByteTrack moved out of `supervision` (removed in 0.30.0) to the
111             # standalone `trackers` package. Its PyPI wheel ships metadata only (no
112             # source ? roboflow/trackers#474), so it MUST come from git. PEP 621 can't
113             # express a git URL, so it's not in pyproject.toml (same pattern as torch).
114             # Installed here so the two D13/P12 tests in tests/test_person_detector.py
115             # actually run in CI instead of skipping via pytest.importorskip.
116             # NOTE: roboflow/trackers uses bare version tags (2.5.0), NOT v-prefixed.
117             run: python -m pip install "trackers @
git+https://github.com/roboflow/trackers.git@2.5.0"
118           - name: Install package (editable) + dev tooling
119             # Editable PEP 621 install: runtime deps + the `dev` group
120             # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
```

```
121             run: python -m pip install -e .[dev]
122           # obsreport (private, soft dep) is absent here: reporting tests skip via
123           # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
124           # is therefore calibrated against THIS lane's number (local runs with
125           # obsreport installed measure a little higher).
126           # Floor calibrated 2026-06-11 against this lane's measured 72% (local
127           # with obsreport: 73%). Ratchet it UP as coverage grows; never down
128           # without an owner decision.
129           - name: Run full suite (with coverage floor)
130             run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
131
132       # Cross-OS determinism guard for the committed golden regression fixtures
133       # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
134       # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
135       # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
136       # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
137       # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
138       golden-crossos:
139         name: Golden fixtures cross-OS (${{ matrix.os }})
140         strategy:
141           fail-fast: false
142           matrix:
143             os: [ubuntu-latest, windows-latest, macos-latest]
144         runs-on: ${{ matrix.os }}
145         timeout-minutes: 15
146         steps:
147           - uses: actions/checkout@v6
148           - uses: actions/setup-python@v6
149             with:
150               python-version: "3.13"
151           - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
152             run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
153           - name: Golden-fixtures characterization (cross-OS determinism guard)
154             # Also runs the freeze-real / time-origin-reset / M2 refusal-gate tests
(synthetic /
155             # tmp_path only ? no GPU/network) so the reset metric-invariance (fix #2) and
the
156             # de-attribution paths get cross-OS coverage and surface any win/mac float
drift.
157             run: |
158               python -m backend.eval.golden_fixtures --check
159               python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
160
161       # Commercial-clean licensing guardrail (MIT/Apache/BSD only ? PACKAGING_PLAN.md P0d).
Scans the
162       # RUNTIME dependency closure (`pip install .` ? no torch/dev) and fails on any
copyleft
163       # (GPL/AGPL/LGPL/SSPL). torch (BSD-3) is installed out-of-band and is not in this
closure. This
164       # automates a guardrail that was human-only; the report is printed for review either
way.
165       # D13/P12 carve-out: `trackers` (Apache-2.0; ByteTrack's new home) is also git-only
and thus
166       # absent from this pyproject closure ? UNguarded here, same shape as torch. It's
Apache-2.0 and
167       # depends only on the still-declared `supervision` (MIT), so the closure is clean
today; if
168       # `trackers` ever pulls a new transitive dep, this gate won't catch it ? audit on
bump.
169       license-scan:
170         name: Dependency license scan (no copyleft in runtime deps)
171         runs-on: ubuntu-latest
```

```
        timeout-minutes: 15
        steps:
          - uses: actions/checkout@v6
          - uses: actions/setup-python@v6
            with:
              python-version: "3.13"
          - name: Install runtime deps + pip-licenses
            run: |
              python -m pip install .
              python -m pip install pip-licenses
          - name: Report dependency licenses
            run: python -m piplicenses --format=markdown --with-urls --order=license
          - name: Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
            run: |
              if python -m piplicenses --format=plain | grep -iE "GPL|SSPL"; then
                echo "::error::A runtime dependency carries a copyleft (GPL/AGPL/LGPL/SSPL)
license ? the guardrail is MIT/Apache/BSD only. Review the report above."
                exit 1
              fi
              echo "OK: no GPL/AGPL/LGPL/SSPL in the runtime dependency closure."

      # Built-wheel smoke (PACKAGING_PLAN.md P0d): build the wheel, install it NON-editable
from a
      # non-repo cwd, and launch the console script. Catches packaging regressions the
source suite
      # can't ? namespace-package resolution (backend has no __init__.py), missing data
files, hidden
      # imports. (Validated locally: a clean-venv wheel install + CPU torch runs `indexer
--help`.)
      build-artifact:
        name: Built wheel smoke (install + launch)
        runs-on: ubuntu-latest
        timeout-minutes: 30
        steps:
          - uses: actions/checkout@v6
          - uses: actions/setup-python@v6
            with:
              python-version: "3.13"
          - name: System libs for opencv-python
            run: |
              # Drop the runner's pre-installed Microsoft apt repo (azure-cli): it
periodically
              # 403s on InRelease and aborts `apt-get update` under the `-e` shell, failing
the
              # build on an unrelated repo we don't use. libgl1/libglib2.0 are in Ubuntu's
archive.
              sudo rm -f /etc/apt/sources.list.d/*microsoft* /etc/apt/sources.list.d/*azure*
              sudo apt-get update
              sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
          - name: Build the wheel
            run: |
              python -m pip install build
              python -m build --wheel
          - name: Install the built wheel + CPU torch (out-of-band index)
            run: |
              python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
              python -m pip install dist/*.whl
          - name: Launch the installed console script from a non-repo dir
            run: |
              cd /tmp
              indexer --help
              python -c "import backend, training, backend.utils.app_home; print('wheel
import OK')"
```

22. user (2026-06-29T10:50:48.409Z)

```
name: Nightly quality regression (Tier-1, offline)

# Tier-1 nightly quality guard. Per-PR `ci.yml` only fires on push/pull_request;
# this runs the SAME offline guards on a schedule so we also catch what per-PR CI
# can't: dependency / transitive-dep drift, flakiness over time, and rot during
# quiet periods when nothing is pushed.
#
# Tier-1 == OFFLINE ONLY: no GPU, no real video, no `run_regression.py` on the
# real corpus (that's Tier-2 ? owner-gated, needs a self-hosted GPU runner +
# the gitignored golden corpus).
#
# Lane parity with ci.yml: every OFFLINE guard job from ci.yml is mirrored here
# (lint · leak-scan · typecheck · import-smoke · full-suite · golden-fixtures ·
# license-scan · build-artifact), with the same Python pin + per-lane pip
# installs so modules collect/run instead of skipping for a missing dep. Two
# intentional differences, NOT drops:
#   - golden-fixtures runs single-OS here (ci.yml::golden-crossos already covers
#     the Win/mac/Linux matrix on every PR; the nightly is about drift-over-time,
#     not cross-OS float drift).
#   - a notify-on-failure job (below) surfaces a SCHEDULED failure as a tracking
#     issue ? ci.yml doesn't need it (a PR/push run already has a red check).
# Lane set + coverage floor are still copied literals, not single-sourced ? see
# the `workflow_call` single-sourcing follow-up in the PR description.
#
# NOTE: scheduled workflows only fire from the repo's DEFAULT branch, so this
# won't actually run on a cadence until it lands on `master`. Use the
# `workflow_dispatch` trigger to run it on demand for validation after merge.

on:
  schedule:
    # 03:00 UTC daily.
    - cron: "0 3 * * *"
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: nightly-${{ github.ref }}
  cancel-in-progress: true

jobs:
  # Lint gate (ruff.toml at repo root) ? mirrors ci.yml::lint.
  lint:
    name: Nightly · Ruff lint
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install ruff
        run: python -m pip install ruff
      - name: Ruff check
        run: ruff check . --output-format github

  # PII / attribution leak guard (tools/leak_scan.py) ? mirrors ci.yml::leak-scan.
  # A structural-rot guard: fails on a committed 32-hex MD5 (the video_id shape)
  # or a personal absolute user path in the shipped-code surface.
  leak-scan:
```

```
62          name: Nightly · Leak scan (PII / attribution guard)
63          runs-on: ubuntu-latest
64          timeout-minutes: 10
65          steps:
66            - uses: actions/checkout@v6
67            - uses: actions/setup-python@v6
68              with:
69                python-version: "3.13"
70            - name: Leak scan
71              run: python -m tools.leak_scan
72
73        # Type gate (mypy.ini at repo root) ? mirrors ci.yml::typecheck. Typed core
74        # deps are installed so mypy sees real signatures (without them
75        # ignore_missing_imports would Any-stub half the checks).
76        typecheck:
77          name: Nightly · Mypy (lenient baseline)
78          runs-on: ubuntu-latest
79          timeout-minutes: 10
80          steps:
81            - uses: actions/checkout@v6
82            - uses: actions/setup-python@v6
83              with:
84                python-version: "3.13"
85            - name: Install mypy + typed core deps
86              run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
87            - name: Mypy
88              run: mypy backend training
89
90        # Syntax sweep + import smoke (no GPU stack) ? mirrors ci.yml::import-smoke.
91        # Guards the "SyntaxError / module-level import error shipping while the suite
92        # stays green" class. Deliberately does NOT install cv2/torch/numpy.
93        import-smoke:
94          name: Nightly · Syntax sweep + import smoke (no GPU stack)
95          runs-on: ubuntu-latest
96          timeout-minutes: 10
97          steps:
98            - uses: actions/checkout@v6
99            - uses: actions/setup-python@v6
100             with:
101               python-version: "3.13"
102           - name: Install minimal deps
103             run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
104           - name: Run import smoke tests
105             run: python -m pytest tests/test_import_smoke.py -v
106
107       # Full offline suite + coverage floor ? mirrors ci.yml::full-suite. CPU-only
108       # torch (no CUDA wheels on a CPU runner), then the editable install pulls the
109       # rest. Floor stays at the ci.yml value (70); never lower it without an owner
110       # decision.
111       full-suite:
112         name: Nightly · Full test suite (offline, mocked)
113         runs-on: ubuntu-latest
114         timeout-minutes: 30
115         steps:
116           - uses: actions/checkout@v6
117           - uses: actions/setup-python@v6
118             with:
119               python-version: "3.13"
120               cache: pip
121           - name: System libs for opencv-python
122             run: |
123               # Drop the runner's pre-installed Microsoft apt repo (azure-cli): it
periodically
```

```
124               # 403s on InRelease and aborts `apt-get update` under the `-e` shell, failing
the
125               # build on an unrelated repo we don't use. libgl1/libglib2.0 are in Ubuntu's
archive.
126               sudo rm -f /etc/apt/sources.list.d/*microsoft* /etc/apt/sources.list.d/*azure*
127               sudo apt-get update
128               sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
129           - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
130             run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
131           - name: Install trackers (git-only dep ? PyPI wheel is broken)
132             # D13/P12: ByteTrack moved to the standalone `trackers` package (git-only;
133             # see ci.yml for the full rationale). Installed here so the D13/P12 tests
134             # run instead of skipping via pytest.importorskip. Bare version tag (no 'v').
135             run: python -m pip install "trackers @
git+https://github.com/roboflow/trackers.git@2.5.0"
136           - name: Install package (editable) + dev tooling
137             run: python -m pip install -e .[dev]
138           - name: Run full suite (with coverage floor)
139             run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
140
141       # Offline quality guard: the committed golden regression fixtures
142       # (docs/GOLDEN_REGRESSION_FIXTURES.md) ? synthetic + de-attributed real
143       # trajectory fixtures, NO video / GPU / DB. Mirrors the offline replay in
144       # ci.yml::golden-crossos (single-OS here; ci.yml already covers cross-OS on
145       # every PR ? nightly is about drift-over-time on the default branch). The
146       # CONTROL replay needs only numpy.
147       golden-fixtures:
148         name: Nightly · Golden fixtures regression (offline quality guard)
149         runs-on: ubuntu-latest
150         timeout-minutes: 15
151         steps:
152           - uses: actions/checkout@v6
153           - uses: actions/setup-python@v6
154             with:
155               python-version: "3.13"
156           - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
157             run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
158           - name: Golden-fixtures characterization (offline)
159             run: |
160               python -m backend.eval.golden_fixtures --check
161               python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
162
163       # Commercial-clean licensing guardrail (MIT/Apache/BSD only) ? mirrors
164       # ci.yml::license-scan. Scans the RUNTIME dependency closure and fails on any
165       # copyleft (GPL/AGPL/LGPL/SSPL). This is exactly the transitive-dep drift the
166       # header cites as a reason the nightly exists ? a new sub-dep can flip a
167       # license between PRs without touching our code.
168       license-scan:
169         name: Nightly · Dependency license scan (no copyleft in runtime deps)
170         runs-on: ubuntu-latest
171         timeout-minutes: 10
172         steps:
173           - uses: actions/checkout@v6
174           - uses: actions/setup-python@v6
175             with:
176               python-version: "3.13"
177           - name: Install runtime deps + pip-licenses
178             run: |
179               python -m pip install .
180               python -m pip install pip-licenses
```

```
          - name: Report dependency licenses
            run: python -m piplicenses --format=markdown --with-urls --order=license
          - name: Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
            run: |
              if python -m piplicenses --format=plain | grep -iE "GPL|SSPL"; then
                echo "::error::A runtime dependency carries a copyleft (GPL/AGPL/LGPL/SSPL)
license ? the guardrail is MIT/Apache/BSD only. Review the report above."
                exit 1
              fi
              echo "OK: no GPL/AGPL/LGPL/SSPL in the runtime dependency closure."

      # Built-wheel smoke (install + launch) ? mirrors ci.yml::build-artifact. Builds
      # the wheel, installs it NON-editable from a non-repo cwd, and launches the
      # console script. Catches packaging regressions the source suite can't.
      build-artifact:
        name: Nightly · Built wheel smoke (install + launch)
        runs-on: ubuntu-latest
        timeout-minutes: 20
        steps:
          - uses: actions/checkout@v6
          - uses: actions/setup-python@v6
            with:
              python-version: "3.13"
          - name: System libs for opencv-python
            run: |
              # Drop the runner's pre-installed Microsoft apt repo (azure-cli): it
periodically
              # 403s on InRelease and aborts `apt-get update` under the `-e` shell, failing
the
              # build on an unrelated repo we don't use. libgl1/libglib2.0 are in Ubuntu's
archive.
              sudo rm -f /etc/apt/sources.list.d/*microsoft* /etc/apt/sources.list.d/*azure*
              sudo apt-get update
              sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
          - name: Build the wheel
            run: |
              python -m pip install build
              python -m build --wheel
          - name: Install the built wheel + CPU torch (out-of-band index)
            run: |
              python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
              python -m pip install dist/*.whl
          - name: Launch the installed console script from a non-repo dir
            run: |
              cd /tmp
              indexer --help
              python -c "import backend, training, backend.utils.app_home; print('wheel
import OK')"

      # Failure visibility ? a silent nightly is useless. Scheduled runs have no PR
      # to redden and nobody is watching the Actions tab on a quiet day (the exact
      # "rot during quiet periods" this workflow exists to catch). On ANY guard
      # failure, open/refresh a single tracking issue so a human is pinged. Uses the
      # built-in GITHUB_TOKEN (no secrets); `issues: write` is scoped to THIS job
      # only ? the guard jobs above keep `contents: read`. Gated on event_name ==
      # 'schedule': a manual `workflow_dispatch` run is being watched, so no issue.
      notify-on-failure:
        name: Nightly · Surface failure (open/refresh tracking issue)
        needs: [lint, leak-scan, typecheck, import-smoke, full-suite, golden-fixtures,
license-scan, build-artifact]
        if: failure() && github.event_name == 'schedule'
        runs-on: ubuntu-latest
        timeout-minutes: 5
```

```
238         permissions:
239           contents: read
240           issues: write
241         steps:
242           - uses: actions/github-script@v7
243             with:
244               script: |
245                 const runUrl =
`${context.serverUrl}/${context.repo.owner}/${context.repo.repo}/actions/runs/${context.runId}`;
246                 const title = 'Nightly quality regression FAILED';
247                 const body = [
248                   'The Tier-1 nightly offline quality guard failed.',
249                   '',
250                   `- Run: ${runUrl}`,
251                   `- Date (UTC): ${new Date().toISOString()}`,
252                   '',
253                   'One of ruff / leak-scan / mypy / import-smoke / full-suite (cov>=70) /',
254                   'golden-fixtures / license-scan / build-artifact regressed.',
255                   'Open the run for the failing lane. Close this issue once green.',
256                 ].join('\n');
257                 // Reuse a single open tracking issue instead of spamming one per night.
258                 const existing = await github.rest.issues.listForRepo({
259                   owner: context.repo.owner,
260                   repo: context.repo.repo,
261                   state: 'open',
262                   labels: 'nightly-failure',
263                   per_page: 1,
264                 });
265                 if (existing.data.length > 0) {
266                   await github.rest.issues.createComment({
267                     owner: context.repo.owner,
268                     repo: context.repo.repo,
269                     issue_number: existing.data[0].number,
270                     body,
271                   });
272                 } else {
273                   await github.rest.issues.create({
274                     owner: context.repo.owner,
275                     repo: context.repo.repo,
276                     title,
277                     body,
278                     labels: ['nightly-failure'],
279                   });
280                 }
281
```

23. assistant (2026-06-29T10:50:51.491Z)

Now let me look at the native-WASB gating detail and check for the documented expected test count (~1487+) in docs or anywhere. Let me also check the GPU/real-video golden gating in test_native_wasb_runner and search docs for the test count.

24. user (2026-06-29T10:50:56.857Z)

```
=== expected test count references ===
docs/CODE_CLEANUP_AUDIT_2026-06-24.md:32:The goal is to fix these systematically, with **zero
behavior change** in Phase 1, and a **strict no-regression gate** (all tests pass, coverage
never decreases).
docs/CODE_CLEANUP_AUDIT_2026-06-24.md:280:- `2026-06-25` ? **P17 done (B2 finish).** The 4
callers that dropped provider error metrics (`trajectory_hybrid.py`, `yolo_hybrid.py`,
`gemini_refine.refine_window`, `gemini_refine.full_video_rallies`) now check `metrics["error"]`
and log loudly instead of silently treating a provider failure as "0 rallies" ? matching the
pattern the `gemini` segmenter already uses. (`yolo_hybrid` was caught during PR #378 review ?
it bound `metrics` but never read it.) Suite 1484/1484 green; ruff + mypy clean. Follow-up:
migrate the hybrids' `process_video` return type from bare `List[Dict]` to the ABC `Result[...]`
```

```
contract (motion.py is already migrated; hybrids suppressed with `type: ignore[override]` for
now).
docs/CODE_CLEANUP_AUDIT_2026-06-24.md:281:- `2026-06-25` ? **P9 done (B1 finish).** Removed
fabricated `random` data from the motion segmenter ? server/receiver/shot_count/avg_speed_kmh ?
`None`, serve/smash/foul/winner events dropped entirely (motion was the only `add_event`
caller). PR #385 review caught a real regression: `run_process_and_stitch.py` crashed on `None
>= min_shot_count` (the `.get(?, 0)` default no longer applied) ? fixed with a None-guard
matching the API path. Suite 1492/1492 green; ruff + mypy clean.
docs/GOLDEN_REGRESSION_FIXTURES.md:133:- Characterization = **behaviour-locking, not
correctness** ? it freezes current behaviour *including current bugs*. "Passing" = "behaviour
unchanged", never "behaviour correct". This caveat must ride in the **test pass/fail message**,
not only the docs.
docs/GOLDEN_REGRESSION_FIXTURES.md:203:- [ ] §6 honest-limits caveat rides in the test pass/fail
messages, not only the docs.
docs/data_pipeline/GOLDEN_SET_IMPLEMENTATION.md:66:- **Branch per slice**, one PR each, tests
green before PR, full suite must stay green.
docs/COMPUTE_DECOUPLED_SERVING/README.md:148:- [ ] **Profile-parity test green** ? truly-local
vs cloud-mock produce byte-identical windows + segment ranges.
run_all_tests.py:60:        + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit
{rc}")
```

25. user (2026-06-29T10:50:57.366Z)

```
=== native_wasb_runner gpu parity gating, unmocked test ===
        NativeWasbConfig(weights_path="/m/w.pth", stream_video=False), tmp_path
    )
    expected_frames = os.path.join(os.path.abspath(str(tmp_path)), f"{key}_frames")
    assert argv[argv.index("--frames_out_dir") + 1] == expected_frames
    rm.assert_called_once_with(expected_frames)


def test_copy_infer_script_copies_byte_identical(tmp_path):
    """Unmocked: the native runner stages the SAME wasb_infer.py the WSL runner runs (the
    byte-parity precondition) into <repo>/src/, byte-for-byte."""
    import filecmp

    from backend.pipeline.detectors import native_wasb_runner as nwr

    r = NativeWasbRunner(NativeWasbConfig(repo_dir=str(tmp_path / "repo")))
    assert r._copy_infer_script() is True
    dest = tmp_path / "repo" / "src" / "wasb_infer.py"
    assert dest.exists() and filecmp.cmp(nwr._INFER_PY, str(dest), shallow=False)


def test_copy_infer_script_false_on_oserror(tmp_path, monkeypatch):
    r = NativeWasbRunner(NativeWasbConfig(repo_dir=str(tmp_path / "repo")))
    monkeypatch.setattr(
        "backend.pipeline.detectors.native_wasb_runner.shutil.copy",
        lambda *a, **k: (_ for _ in ()).throw(OSError("no space")),
    )
=== count test functions actually defined ===
1390
=== count async test functions ===
0
```

26. assistant (2026-06-29T10:51:01.770Z)

The documented test count is in docs/CODE_CLEANUP_AUDIT_2026-06-24.md (1484/1484 and 1492/1492 green, dated 2026-06-25). The raw `def test_` count is 1390 but parametrize multiplies the actual collected count. Let me try to get the actual collected count and verify the licensing/owner-gated guardrails. Let me also confirm the GPU byte-parity gate documentation for native-WASB.

27. user (2026-06-29T10:51:11.506Z)

```
  /home/avidullu/projects/khelsutra-guru/.venv/lib/python3.12/site-
packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with
`starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
1513 tests collected in 5.38s
EXIT: 0
```

28. user (2026-06-29T10:51:12.901Z)

```
=== latest test-count mentions across docs (sorted) ===
1365 pass
1390 pass
1463 green
1463 pass
1463/1463 green
1484 green
1492 green
Suite 1068
Suite 1105
Suite 1114
Suite 1126
Suite 1463
Suite 1484
Suite 1492

=== README test count ===
157:Output lands in `--output-dir` (default `output/`) unless `--trim-output` is an absolute
path. This is the fastest way to *see* the watermark on a real clip ? set
`stitching.watermark.enabled: false` in `config.json` to turn it off.
189:    "ai_model": "gemini-flash-latest",
240:*   `validation.min_blur_threshold` ? defaults to `20.0` (deliberately permissive for noisy
indoor-arena GoPro footage; see `docs/archives/TEST_RUN_SUMMARY.md` for why).
278:### Q6: How can I quickly test the indexing without waiting to parse an entire multi-GB
video file?
279:You can pass the `--max-frames <N>` argument in CLI or `max_frames` in the POST API payload.
This restricts the segmenter frame loop to process only the first $N$ sub-sampled frames,
enabling you to test motion filters and court relevance checks in seconds:
319:### Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without
uploading a huge multi-GB file?
376:## ? Running the Test Suite
377:The test suite uses `pytest` and lives in `tests/`. From the project root:
380:pip install pytest pytest-cov --user
382:# One-liner full suite (recommended before opening a PR) ? discoverable wrapper that
```

29. assistant (2026-06-29T10:51:16.168Z)

I now have the actual collected count (1513 in the current local checkout, with obsreport/cv2/etc. present). The documented latest figure is 1492/1492 (2026-06-25). The "~1487+" in the prompt aligns with these mid-1480s/1490s documented figures. I have everything I need. Let me compile the structured output.

30. user (2026-06-29T10:52:25.164Z)

Structured output provided successfully